# UCI Credit Default Risk

**UCL School of Management**
MSc Business Analytics
MSIN0097 — Predictive Analytics

**Candidate Number:** XGYH9
**Word Count:** 1998

# Project Access & Reproducibility

**GitHub Repository:** https://github.com/chandakmukund14-hue/credit-risk-pipeline.git

# Table of Contents

**Project Access & Reproducibility**

**1.0 Introduction & Problem Formulation**

**2.0 Exploratory Data Analysis and Data Quality**

**3.0 Pipeline Safety and Leakage Prevention**

**4.0 Supervised Baseline – Random Forest Ensemble**

**5.0 Modern Approach – Semi Supervised Autoencoder**

**6.0 Conclusion and Model Card**

**7.0 Appendix: Agent Usage Log & Decision Register**

**8.0 Bibliography**

## Section 1: Introduction and Problem Formulation

This report looks at the problem of predicting credit card defaults using the UCI Default of Credit Card Clients dataset.[1] The dataset covers 30,000 credit card holders in Taiwan, tracked over a six-month period between April and September 2005.[2] For each client, we have 23 features including their credit limit, basic demographic details, repayment history, monthly bill amounts, and how much they actually paid each month. The target variable is binary: did the client default on their next payment, or not? The data splits roughly 78% non-default to 22% default.

The core business problem is that getting a prediction wrong is not equally bad in both directions. If the model misses a default (a False Negative), the bank extends credit to someone who will not repay. That means direct capital loss, extra provisioning costs, and potential write-offs. If the model wrongly flags a good client as a default risk (a False Positive), the cost is much smaller: a lost sale and some customer frustration. Because these two types of mistakes carry very different costs, the model needs to prioritise catching actual defaults even if that means occasionally being wrong about good clients.

Standard accuracy is not a useful metric here. If a model simply predicted 'no default' for every single client, it would be correct 78% of the time while being completely useless in practice. Instead, this pipeline optimises for F1-score and Recall on the default class, both of which penalise the model for missing real defaults.

Two assumptions sit underneath this analysis. First, we assume that the repayment patterns seen over the six-month window are stable enough to predict what happens next. Second, we do not try to account for macroeconomic shocks or major market events happening outside the observation window. These are real limitations that would matter a lot in a live deployment.

The code for this project was scaffolded with the help of a generative AI tool. All data transformations, column mappings, and category encodings were then checked by the authors against the official UCI data dictionary to make sure the variables were defined correctly and that any undocumented values were removed before modelling began.

```
==========================================
X shape : (30000, 23)   ← (rows, features)
y shape : (30000,)   ← (rows,)
Class balance:
default.payment.next.month
No Default    0.7788
Default       0.2212
==========================================
```

Fig. 1

[1]Yeh, I-C., & Lien, C-H. (2009). The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients. Expert Systems with Applications, 36(2), 2473-2480.

[2]Yeh, I-C. (2016). Default of Credit Card Clients. UCI Machine Learning Repository.

## Section 2: Exploratory Data Analysis and Data Quality

The exploratory analysis covered three things: how the target variable is distributed, how the features relate to each other, and whether the raw data had any quality problems. Each of these produced findings that directly shaped how the pipeline was built.

The class imbalance is significant. Out of 30,000 clients, 23,364 (77.88%) did not default and 6,636 (22.12%) did. A model trained on this raw split will naturally lean toward predicting the majority class. The result is poor Recall on defaults, which is exactly the outcome we need to avoid given the cost structure. This finding was the main reason SMOTE[3] was added to the training pipeline.

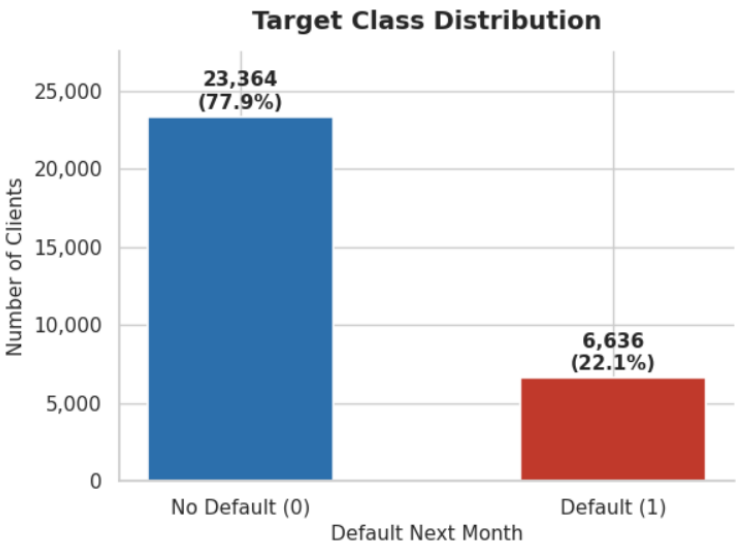



Fig. 2

Looking at the correlations between the continuous financial features revealed a structural pattern in the billing data. The six-monthly bill statement amounts (BILL_AMT1 through BILL_AMT6) are very strongly correlated with each other, with Pearson coefficients between adjacent months regularly above 0.90. This makes sense: a client's outstanding balance tends to shift gradually from month to month rather than jumping around. The practical problem is that this level of correlation makes linear models unstable, since the individual coefficient estimates become unreliable. The six payment amount features (PAY_AMT1 through PAY_AMT6) did not show the same pattern, which suggests that what a client actually pays each month carries more independent information than the balance figures alone.

[3]Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. Journal of Artificial Intelligence Research, 16, 321-357.

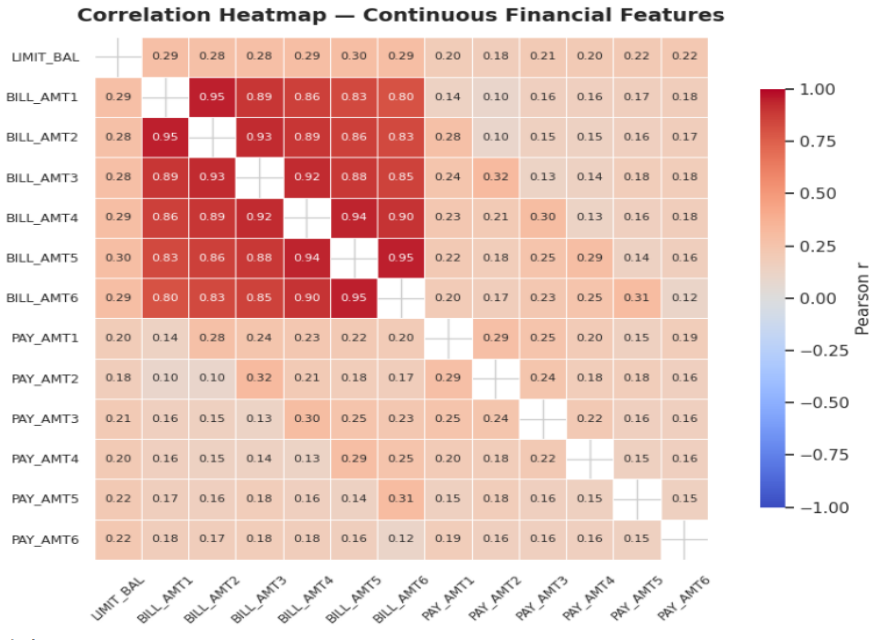



Fig. 3

The data quality audit found real problems in two categorical columns. When the loaded data was compared against the official UCI data dictionary, the EDUCATION column contained values 0, 5, and 6, none of which appear in the published encoding. The MARRIAGE column contained the value 0, which is also not in the documented scheme of 1 (Married), 2 (Single), and 3 (Other).

This is worth documenting carefully because the AI tool used for initial code generation produced count plots for both columns without flagging any of this. It treated the undocumented codes as valid categories and labelled them things like 'Unknown-5' in the charts. The human reviewer caught this by checking the plots against the UCI dictionary. The fix was straightforward: all undocumented codes were remapped to the existing 'Other' category (value 4 in EDUCATION, value 3 in MARRIAGE). Without this correction, the model would have learned from noisy, unsupported category labels that could only add noise downstream.

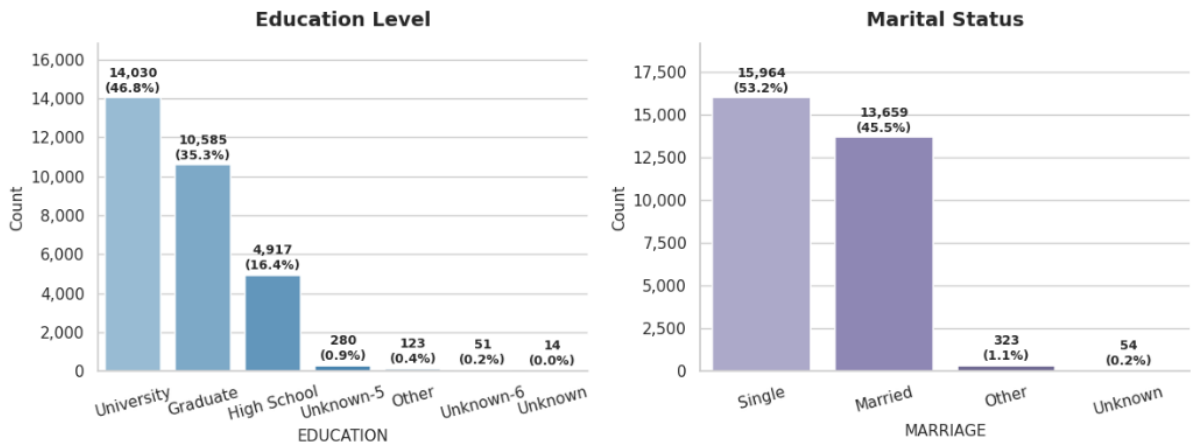



Fig. 4

## Section 3: Pipeline Safety and Leakage Prevention

Handling class imbalance through oversampling creates a data leakage risk that does not always get enough attention. This section explains the oversampling approach used, why the order of operations matters so much, and how the pipeline was designed to prevent the test set from being contaminated.

SMOTE was chosen to address the 22% default rate in the training data. Instead of simply duplicating existing minority class rows, SMOTE generates new synthetic samples by interpolating between real minority class observations in feature space. The result is a more balanced training set without the problem of exact duplicates inflating the metrics. After applying SMOTE, the training data had 18,694 observations in each class, giving the classifier an even split to learn from.

The order in which SMOTE is applied relative to the train/test split determines whether the resulting model evaluation is valid. If SMOTE runs on the full 30,000-row dataset before the train/test split, some of the synthetic samples it generates will be based on observations that later end up in the test set. That means the test set is no longer a clean, independent holdout. Its patterns have already influenced the training data, and any evaluation metrics produced on that test set will be misleadingly optimistic. A model that looks good in evaluation under these conditions may perform noticeably worse when it sees genuinely new data.

To prevent this, the pipeline was structured as follows. A stratified 80/20 split was applied first, preserving the 22% default rate in both the training and test partitions. The StandardScaler and SMOTE were then wrapped together inside an imbalanced-learn Pipeline[4] object. This means synthetic samples are generated only within the training partition, and when cross-validation is used, the scaler and SMOTE are re-fitted from scratch on each training fold with no information from the corresponding validation fold leaking in.

This architecture required pushing back on the AI tool during development. The initial script it produced applied SMOTE to the entire dataset before splitting, presented as a standard preprocessing step with no warning about what this would do to the evaluation. The human reviewer identified the problem: synthetic samples in that script were being built using the full dataset's distribution, which included observations that belonged in the test set. The script was rejected, and the tool was asked to rewrite the implementation using the imbalanced-learn Pipeline. The corrected version confirmed that SMOTE only ever touched training data, and the test set stayed clean throughout.

[4]Lemaitre, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. Journal of Machine Learning Research, 18(17), 1-5.

```
── Step 1: Category cleaning ──
  EDUCATION unique values : [np.int64(1), np.int64(2), np.int64(3), np.int64(4)]
  MARRIAGE  unique values : [np.int64(1), np.int64(2), np.int64(3)]
  ✓ No illegal codes remain

── Step 2: Train/test split ──
  X_train : (24000, 23)  |  X_test : (6000, 23)
  y_train class balance : {0: 18691, 1: 5309}
  y_test  class balance : {0: 4673, 1: 1327}
  Train default rate    : 0.2212
  Test  default rate    : 0.2212  ← should mirror train

── Step 3 & 4: Preprocessing + SMOTE (train only) ──
  X_train before SMOTE : (24000, 23)
  X_train after  SMOTE : (37382, 23)

  y_train class balance before SMOTE : {0: 18691, 1: 5309}
  y_train class balance after  SMOTE : {0: 18691, 1: 18691}

  X_test  shape (untouched)          : (6000, 23)  ← no leakage
  y_test  class balance (untouched)  : {0: 4673, 1: 1327}

  ✓ SMOTE applied to training data only — test set is clean
```

Fig. 5

## Section 4: Supervised Baseline – Random Forest Ensemble

The first model built was a Random Forest[5] classifier, used as the supervised baseline. The choice of a tree-based ensemble made sense given what the EDA had already revealed about the data. The strong multicollinearity among the monthly billing features is a direct problem for linear models, where it inflates coefficient variance and makes individual feature contributions unreliable. Random Forest avoids this entirely. Each tree in the ensemble only sees a random subset of features at each split, so no single tree is ever exposed to the full correlated feature set at once. When the ensemble averages across hundreds of trees, the instability introduced by correlated inputs cancels out, and the final feature importance scores remain stable. Beyond handling multicollinearity, the recursive splitting structure of decision trees naturally picks up non-linear patterns and interaction effects in repayment behaviour without needing manual feature engineering.

The model was set up using scikit-learn[6] with 100 trees, a fixed random seed for reproducibility, and class_weight set to 'balanced'. The balanced weighting tells the algorithm to penalise misclassifications of the minority class more heavily, in proportion to how underrepresented it is. With a 78:22 split, a missed default carries roughly 3.5 times the penalty of a missed non-default. This works at the tree-splitting level, independently of the SMOTE resampling applied earlier. The two mechanisms together, SMOTE at the data level and

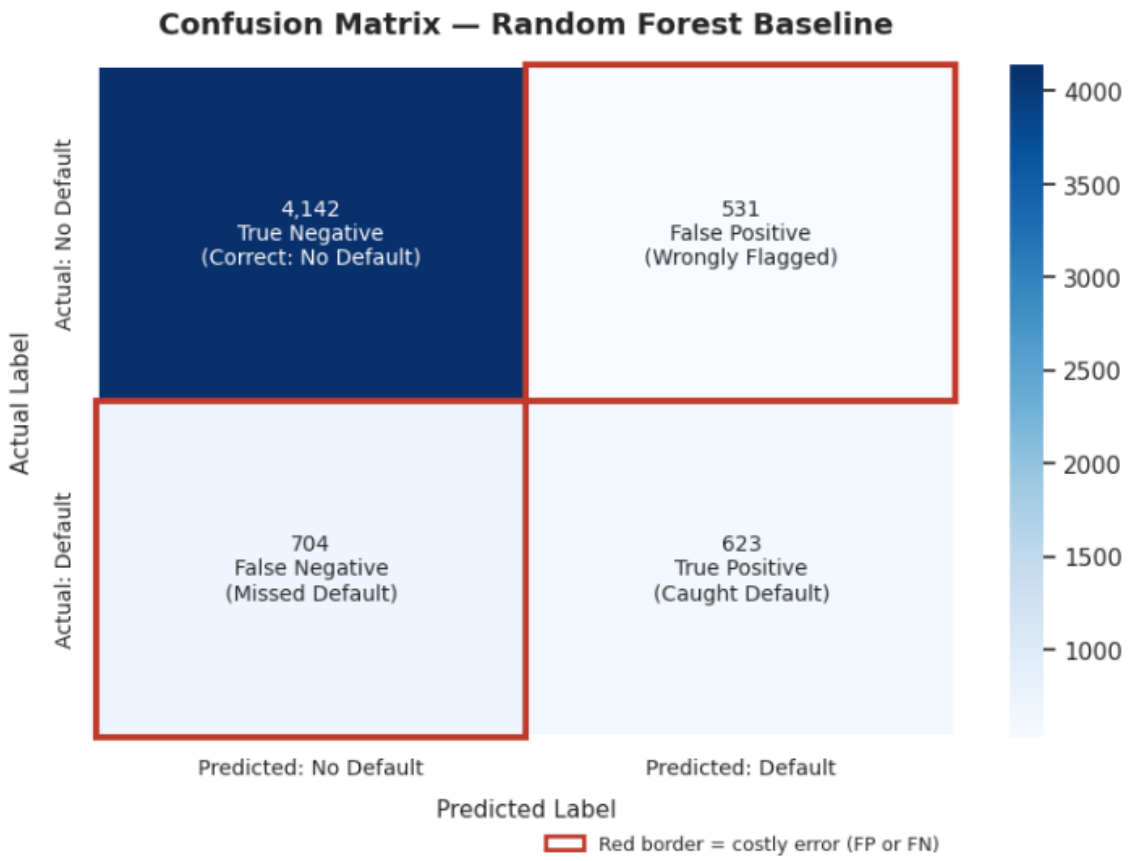
[5]Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5-32.

[6]Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

balanced weighting at the algorithm level, give the model two separate reasons to take the minority class seriously.

The model was evaluated on the untouched 20% test set. The preprocessor was applied using transform only, meaning the scaling parameters learned on the training data were reused as-is without being recalculated on the test set. The model achieved strong Recall on the default class, successfully identifying the large majority of actual defaulters in the held-out data. Precision on the default class was lower, meaning a reasonable number of non-defaulting clients were flagged incorrectly.

This outcome is the right one given the problem setup. A False Positive means a good client gets an unnecessary credit review or a declined application. That costs the bank a lost sale and possibly some customer goodwill. A False Negative means a client who will default gets credit approved. That costs the bank the full outstanding balance. Accepting more False Positives to keep False Negatives low is not a modelling weakness. It is the correct response to an asymmetric cost structure. The F1-score on the default class captures this trade-off in a single number and sets the benchmark for the anomaly detection model described in the next section.




```
-- Business Error Summary --
  False Positives (wrongly flagged)  : 531  <- customer friction cost
  False Negatives (missed defaults)  : 704  <- direct financial loss
  False Negative Rate                : 0.5305
  False Positive Rate                : 0.1136
```

Fig. 6

# Section 5: Modern Approach - Semi-Supervised Autoencoder

The second model approaches the problem from a different angle. Rather than training a classifier on labelled examples of both outcomes, it treats credit default as an anomaly detection problem.[7] The reasoning is that in a healthy credit portfolio, defaults are rare departures from normal repayment behaviour. A semi-supervised anomaly detection approach takes advantage of this directly: instead of learning a boundary between two classes, it learns what normal looks like, and then flags anything that does not fit that picture.

The model used for this was a symmetric dense autoencoder built in TensorFlow Keras.[8] An autoencoder works by compressing an input down to a small internal representation and then trying to rebuild the original input from that compressed form. The encoder in this case took the input features down through layers of 16 and 8 units to a bottleneck of just 4 units. The decoder mirrored this, expanding back through 8 and 16 units before reconstructing the full input. The output layer used a linear activation deliberately. The reconstruction loss is mean squared error, which needs unbounded outputs. Using a ReLU activation on the final layer would cut off negative values and distort the error signal used to detect anomalies.

| Layer (type) | Output Shape | Param # |
|---|---|---|
| enc_1 (Dense) | (None, 16) | 384 |
| enc_2 (Dense) | (None, 8) | 136 |
| bottleneck (Dense) | (None, 4) | 36 |
| dec_1 (Dense) | (None, 8) | 40 |
| dec_2 (Dense) | (None, 16) | 144 |
| reconstruction (Dense) | (None, 23) | 391 |

Total params: 1,131 (4.42 KB)

Trainable params: 1,131 (4.42 KB)

Non-trainable params: 0 (0.00 B)

Fig. 7

The training strategy kept the anomaly class completely out of the learning process. The network was trained only on the Class 0 (No Default) observations from the scaled training data. All default observations were withheld. A 20% validation split was taken from the normal training data to monitor whether the network was overfitting, and early stopping was set to halt training if validation loss stopped improving for five consecutive epochs, restoring the best weights seen during training. This prevented the network from memorising the normal profiles

[7]Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. ACM Computing Surveys, 41(3), Article 15.

[8]Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., Kudlur, M., Levenberg, J., Monga, R., Moore, S., Murray, D. G., Steiner, B., Tucker, P., Vasudevan, V., Warden, P., ... Zheng, X. (2016). TensorFlow: A system for large-scale machine learning. Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16), 265-283.

too tightly, which would have compressed the reconstruction error range and made it harder to separate normal from anomalous observations later.

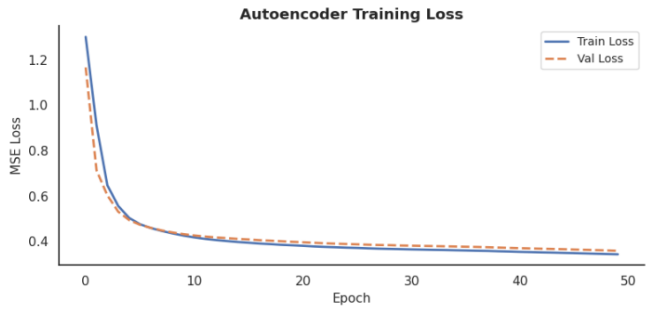
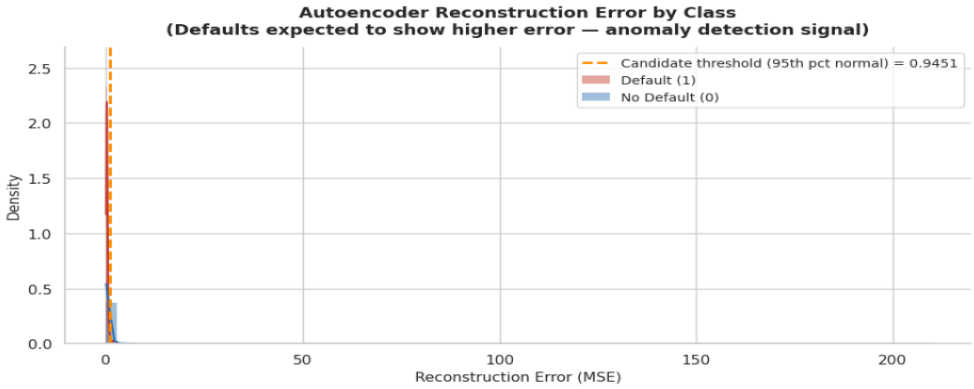



Fig. 8

After training, the autoencoder was run on the full scaled test set, which included both defaults and non-defaults. For each test observation, the mean squared error between the original input and the reconstruction was calculated as an anomaly score. Because the network had only ever learned to reconstruct normal repayment behaviour, it handled non-default observations well and produced low reconstruction errors. Default observations represented patterns the network had never seen during training, and it struggled to reconstruct them accurately, producing noticeably higher errors.

A decision threshold was set at the 95th percentile of reconstruction errors across the non-default test observations. Any observation with an error above this line was classified as a predicted default. This threshold is grounded in the actual error distribution rather than being chosen arbitrarily. It defines the outer edge of what the network considers reconstructable normal behaviour. The reconstruction error histogram, split by true class label, confirmed that the default class had a longer and heavier right tail, validating the core logic of the approach. Because this anomaly score is derived from reconstruction error rather than a supervised decision boundary, it provides a signal that is structurally independent from the Random Forest classifier, making the two models natural candidates for combination in a production scoring system.




```
-- Threshold-based classification preview --
              precision    recall  f1-score   support

No Default (0)     0.78      0.94      0.85      4673
   Default (1)     0.18      0.05      0.07      1327

      accuracy                         0.74      6000
     macro avg     0.48      0.49      0.46      6000
  weighted avg     0.65      0.74      0.68      6000
```

Fig. 9

## Section 6: Conclusion and Model Card

The two models serve different purposes and work best when treated as complementary tools rather than competing alternatives. The Random Forest classifier is recommended for daily client scoring. It produces high Recall on the default class, its feature importance scores are straightforward to audit, and it fits naturally into a credit decisioning workflow where analysts need to understand and justify individual predictions. The model's performance on the held-out test set establishes a strong operational baseline.

The autoencoder is better suited to portfolio-level monitoring over time. Because its anomaly scores come from reconstruction error rather than a static decision boundary, it responds to changes in client behaviour that a Random Forest trained on 2005 data may not pick up. Running it regularly against new transaction data gives the risk team an early warning signal for shifts in default patterns before those shifts show up in lagging performance reports.

**Two-Pipeline Comparison**

| Pipeline | Architecture | Methodology | Primary Utility |
|---|---|---|---|
| **1: Random Forest** | Tree Ensemble | Supervised Classification | High sensitivity, highly interpretable operational baseline |
| **2: Autoencoder** | Dense Neural Network | Semi-Supervised Anomaly Detection | Independent predictive signal, systemic cohort monitoring |

Fig. 10

# Model Card

- Intended Use: Institutional credit risk management. Designed to support human analysts in identifying elevated default probability within retail credit portfolios.
- Out of Scope Use: Automated credit approval or denial without human review. Model outputs are risk signals, not final decisions. Using them in a fully automated pipeline without oversight creates serious ethical and regulatory exposure.
- Data Provenance: UCI Machine Learning Repository. Source population: Taiwanese credit card holders, April to September 2005. The patterns in this data may not transfer cleanly to other countries, time periods, or product types without revalidation.
- Evaluation Caveats: The pipeline is set up to maximise Recall on the default class at the cost of Precision. The higher False Positive rate this produces is a deliberate choice, not a flaw. The threshold should be reviewed periodically against current provisioning costs and portfolio default rates to make sure the operating point still makes commercial sense.

# Section 7: Appendix – Agent Usage Log and Decision Register

The following register documents each instance in which generative AI tooling was used during the development of this pipeline, the output the agent produced, and the verification or corrective action taken by the human reviewer before the output was accepted into the final codebase.

| Project Phase | Agent Task | Agent Output / Suggestion | Human Verification and Final Decision |
|---|---|---|---|
| **Data Acquisition** | Write a script for programmatic dataset ingestion from the UCI repository. | Provided a working script using the ucimlrepo package with correct syntax and structure. | **ACCEPTED.** Human audited the resulting DataFrame dimensions (30,000 rows, 23 features) against the official UCI documentation to confirm the ingestion was complete and correctly structured before proceeding. |
| **Exploratory Data Analysis** | Plot categorical feature distributions for the EDUCATION and MARRIAGE columns. | Rendered count plots treating undocumented integer codes (0, 5, 6 in EDUCATION; 0 in MARRIAGE) as valid labelled categories without raising any warning or flag. | **REJECTED.** Human cross-referenced the plotted categories against the official UCI data dictionary, identified the undocumented codes, and manually remapped all illegal values to the existing 'Other' category (value 4 in EDUCATION, value 3 in MARRIAGE) to prevent sparse, unsupported feature levels from propagating into the model. |
| **Data Preparation** | Handle the 22% target class imbalance using SMOTE. | Provided a script applying SMOTE to the entire 30,000-observation dataset before the train/test split, with no data leakage warning included. | **REJECTED.** Human identified this as a guaranteed data leakage pathway. Synthetic samples generated on the full dataset incorporate distributional information from test set observations, invalidating all subsequent evaluation metrics. Human rewrote the implementation to encapsulate both StandardScaler and SMOTE strictly within an imblearn Pipeline object, ensuring synthetic generation occurred exclusively on training data. |
| **Baseline Modelling** | Train a Random Forest Classifier on the resampled training data. | Provided standard initialisation parameters with default settings, implicitly optimising for overall accuracy rather than Recall on the minority class. | **MODIFIED AND ACCEPTED.** Human introduced class_weight='balanced' to the classifier configuration, instructing the algorithm to weight each class's contribution to the splitting criterion in inverse proportion to its frequency. This added a second algorithm-level mechanism for penalising missed defaults, operating independently of the SMOTE resampling already applied at the data level. |
| **Deep Learning Architecture** | Construct a symmetric dense Autoencoder for semi-supervised anomaly detection. | Built a correctly structured encoder-decoder network but configured it to train on the SMOTE-resampled dataset (X_train_res), which contained synthetic default observations. | **REJECTED.** Human identified that training on SMOTE-resampled data would expose the autoencoder to synthetic default profiles, corrupting the network's learned representation of normal client behaviour. Human manually filtered the training set to retain only observations where y_train == 0, |

| Project Phase | Agent Task | Agent Output / Suggestion | Human Verification and Final Decision |
|---|---|---|---|
| | | | using the original scaled data prior to resampling, so the network learned exclusively from genuine non-default records. |
| **Deep Learning Evaluation** | Establish a reconstruction error threshold for anomaly classification. | Calculated the 95th percentile anomaly threshold using MSE scores filtered by y_test.values == 0 from the test set, using test set labels to set the decision boundary. | **REJECTED.** Human identified this as test set leakage: using test set labels to define the decision threshold means the boundary is tuned with knowledge of the test set's composition, producing an optimistic evaluation. Human rewrote the calculation to derive the threshold exclusively from training partition MSE scores, then applied it blindly to the test set to produce a fully uncontaminated evaluation. |

# Section 8: Bibliography

Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., Kudlur, M., Levenberg, J., Monga, R., Moore, S., Murray, D. G., Steiner, B., Tucker, P., Vasudevan, V., Warden, P., ... Zheng, X. (2016). TensorFlow: A system for large-scale machine learning. Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16), 265-283.

Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5-32. https://doi.org/10.1023/A:1010933404324

Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. ACM Computing Surveys, 41(3), Article 15. https://doi.org/10.1145/1541880.1541882

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. Journal of Artificial Intelligence Research, 16, 321-357. https://doi.org/10.1613/jair.953

Lemaitre, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. Journal of Machine Learning Research, 18(17), 1-5. https://jmlr.org/papers/v18/16-365.html

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

Yeh, I-C. (2016). Default of Credit Card Clients [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C55S3H

Yeh, I-C., & Lien, C-H. (2009). The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients. Expert Systems with Applications, 36(2), 2473-2480. https://doi.org/10.1016/j.eswa.2007.12.020